

■ 2.5.10 Advanced Topic: Function Arguments and Local Variables

In a symbolic language like *Mathematica*, the exact treatment of function arguments and local variables can be quite complicated.

The first issue to understand is how the argument x is treated in a function definition like $f[x_] := x^2$.

Mathematica uses an approach known as *substitution semantics*. When it evaluates $f[5]$, *Mathematica* simply takes the expression x^2 that forms the “body” of the function and substitutes 5 for x everywhere in it. The procedure is effectively to evaluate $f[5]$ using the replacement $x^2 /. x \rightarrow 5$.

An alternative approach that *Mathematica* does not use would be temporarily to *assign* the function argument x to have value 5, and then to evaluate the body of the function with this assignment made.

The substitution semantics approach that *Mathematica* uses has several advantages. One of them is that it allows you to use the values of function arguments even in places where they would not usually be evaluated. This means, for example, that you can give as a function argument an object that appears in the body of the function on the left-hand side of an assignment, or inside the body of a pure function. The appropriate value is substituted for every occurrence of the function argument in the body of the function, whether or not it would be evaluated in that position.

One of the key issues in the treatment of function arguments is how significant the actual *names* of function arguments are. An advantage of substitution semantics is that it makes these actual names largely irrelevant. If *Mathematica* instead used temporary assignment, the names could be significant, and many problems would arise. For example, with a definition like $f[x_] := x^2$, an expression such as $f[1+x]$ would be difficult to evaluate. The first step in the evaluation would effectively be the assignment $x = 1+x$. If x had no explicit value defined, then this assignment would immediately lead to an infinite loop. Substitution semantics avoids these difficulties, by making just one pass through the body of the function, inserting the values given for the function arguments.

Another issue about the treatment of function arguments is the form in which their values are “passed” to the body of the function. In a standard *Mathematica* function that evaluates its arguments, it is just the values of the arguments that are passed to the body of the function. However, if you set a *Mathematica* function which does *not* evaluate certain arguments, then these arguments will be passed in exactly the form that you specify them.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

g is defined to assign a to have value b . `In[1]:= g[a_, b_] := (a = b)`

This does the assignment $x = y$. `In[2]:= g[x, y]`
`Out[2]= y`

The first step here is to evaluate x to give y . Then y is substituted for each occurrence of a in the body of g . The final assignment done is $y = z$. `In[3]:= g[x, z]`
`Out[3]= z`

The value of x is unchanged, but y now evaluates to z . `In[4]:= {x, y}`
`Out[4]= {z, z}`

This specifies that none of the arguments to h are to be evaluated. `In[5]:= Attributes[h] = HoldAll`

Here is the same definition as we used for g . `In[6]:= h[a_, b_] := (a = b)`

Now x is not evaluated before being passed to the function body. The final assignment that is done is therefore $x = 2$. `In[7]:= h[x, 2]`
`Out[7]= 2`

One point to remember when you set up functions in *Mathematica* is that the values of arguments are substituted *wherever* objects with the same name as those arguments appear in the body of the function. Thus, for example, you have to be careful that the names of local variables you use inside a **Block**, or in an iteration function like **Do**, **Table** or **Sum**, do not conflict with the names of the function arguments. (You must also avoid conflicts with arguments to functions that you define inside your main function.)

In general, the way that local variables are handled in *Mathematica* is as follows. Whenever a block is entered, the previous values of the variables which you declare to be local are effectively pushed on a stack. When the block is finished, the original values of the variables are restored.

When you are inside a **Block** with a local variable named x , there is no way for you to access a value that x has outside the block.

One complication occurs if you *return* the local variable x from a block, without having assigned it a value inside the block. In this case, *Mathematica* treats the returned expression as containing an object with name x . When the previous value of x is restored, evaluation of the object named x will give this previous value.

A final issue concerns a complicated interaction between the symbolic nature of

Mathematica, and the treatment of function arguments and local variables.

This function sums the value of the object `x` when `i` runs from 1 to 3.

```
In[8]:= s[x_] := Sum[x, {i, 3}]
```

The expression `j^2` is independent of `i`, and so the result is just `3 j^2`.

```
In[9]:= s[j^2]
```

```
Out[9]= 3 j^2
```

The variable `i` that appears in the argument is now “captured” by `Sum`. The result is the value of `Sum[i^2, {i, 3}]`.

```
In[10]:= s[i^2]
```

```
Out[10]= 14
```

The “capture” of symbolic function arguments by local variables in the body of a function can have both good and bad consequences. Sometimes it can be very confusing to have the name of a local variable conflict with a value you give for a function argument.

The best way to solve this problem is somehow to have the names of the local variables that appear in your functions be such that they will never appear in the values of function arguments. You can conveniently achieve the kind of “static scoping” that is needed by setting up “contexts” for symbol names, as discussed in Section 2.7.